

Readable Async Workflows in Modern C++

Coroutines Meet `std::expected`

FUTONG LIU

Readable Async Workflows in Modern C++ Coroutines Meet `std::expected`

CppCon 2026
September 16, 2026

Futong Liu
Software Engineer

TechAtBloomberg.com

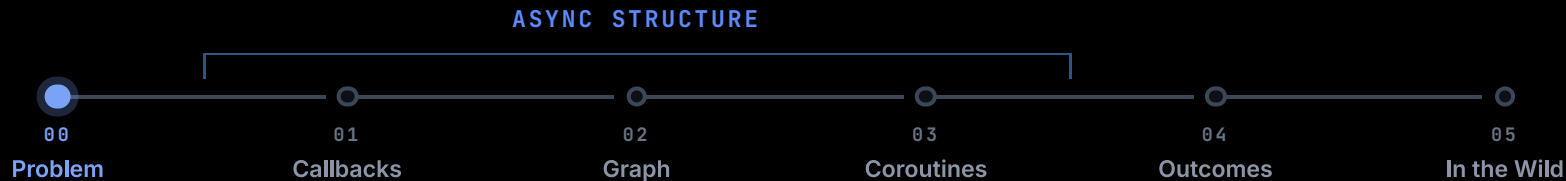
© 2026 Bloomberg Finance L.P. All rights reserved.

Engineering

Bloomberg

Part 0 · Problem

An example async workflow



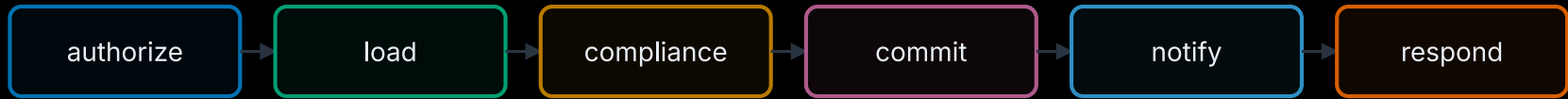
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

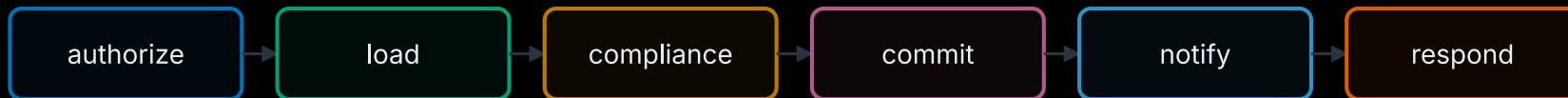
Bloomberg

Engineering

An example: One workflow with six steps



An example: One workflow with six steps



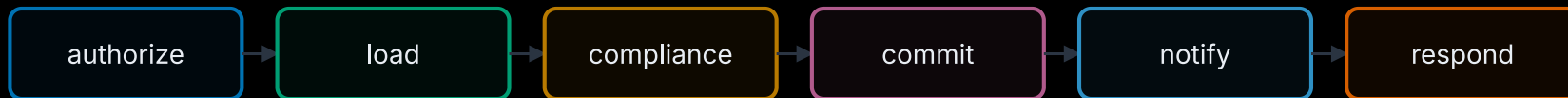
```
Response activateOrders(const Request& request)
{
    authorize(request);

    const std::vector<Order> orders = loadOrders(request.orderIds);
    const std::vector<Order> checked = checkCompliance(orders);

    const std::vector<CommitResult> committed = commit(checked);
    notifyDownstream(committed);

    return makeResponse(committed, {});
}
```

An example: One workflow with six steps



```
Response activateOrders(const Request& request)
{
    authorize(request);

    const std::vector<Order> orders = loadOrders(request.orderIds);
    const std::vector<Order> checked = checkCompliance(orders);

    const std::vector<CommitResult> committed = commit(checked);
    notifyDownstream(committed);

    return makeResponse(committed, {});
}
```

Business ordering matches source-code ordering.

Why make this workflow async?



Why make this workflow async?

■ doing work ▨ waiting on I/O / downstream

OPERATION LIFETIME



Most of each request's lifetime is spent waiting on I/O.

Why make this workflow async?

■ doing work ■ waiting on I/O / downstream

OPERATION LIFETIME



THREAD OCCUPANCY



Synchronous wait: concurrent, but thread is waiting for operation.

Why make this workflow async?

■ doing work ■ waiting on I/O / downstream

OPERATION LIFETIME



THREAD OCCUPANCY



Asynchronous wait: the operation waits without making a thread wait.

Why make this workflow async?

■ doing work ■ waiting on I/O / downstream

OPERATION LIFETIME



THREAD OCCUPANCY



Asynchronous wait: the operation waits without making a thread wait.

Threads already give us concurrency.

Async makes that concurrency cheaper.

Fewer blocked threads · More operations in flight · Higher throughput

What makes a workflow readable?

A workflow anatomy for reasoning about async readability.

In synchronous code, the call stack implicitly carries sequencing, state lifetime, and completion.

Once work can outlive the call that started it, that complexity still has to be represented somewhere.

DIMENSION	QUESTION	WHAT TO LOOK FOR
Control flow	What happens next?	Where is the next step encoded?
Data flow	What must survive the wait?	Where does the state live?
Completion	When is the work done?	What signals completion?



What makes a workflow readable?

A workflow anatomy for reasoning about async readability.

In synchronous code, the call stack implicitly carries sequencing, state lifetime, and completion.

Once work can outlive the call that started it, that complexity still has to be represented somewhere.

DIMENSION	QUESTION	WHAT TO LOOK FOR
Control flow	What happens next?	Where is the next step encoded?
Data flow	What must survive the wait?	Where does the state live?
Completion	When is the work done?	What signals completion?

Readable orchestration makes all three answers easy to find.



Where we're going

ASYNC STRUCTURE

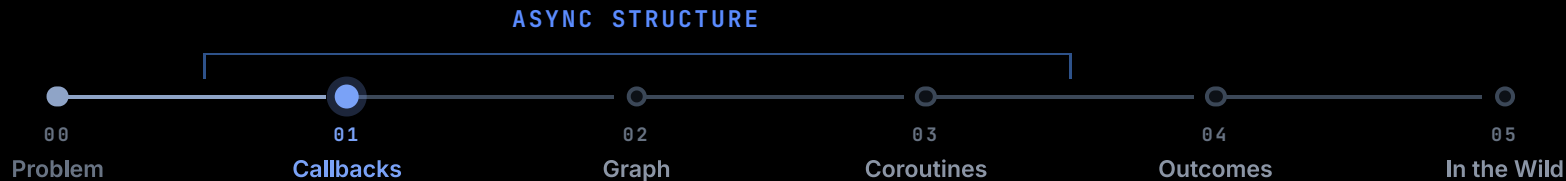
control · data · completion

00	01	02	03	04	05
Problem	Callbacks	Graph	Coroutines	Outcomes	In the Wild
Where does workflow structure go?				How do we represent outcomes?	Can change stay local?



Part 1 · Callbacks

What happens when we write the continuation ourselves?



What does a callback look like?

Put "what happens next" in another function.

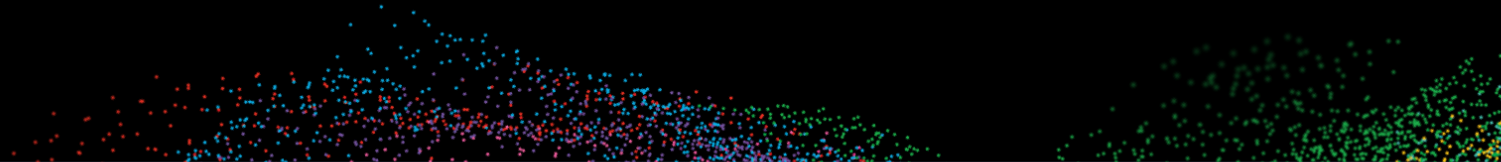


What does a callback look like?

Put “what happens next” in another function.

ONE ORDER · COMMIT → NOTIFY

```
int rc = commitAsync(order, onCommitted);  
                        ^^^^^^^^^^^^^^^  
                        callback
```



What does a callback look like?

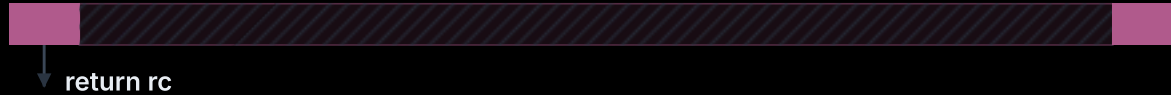
Put "what happens next" in another function.

ONE ORDER · COMMIT → NOTIFY

```
int rc = commitAsync(order, onCommitted);  
                        ^^^^^^^^^^^^^^^  
                        callback
```

issue I/O

operation completes



What does a callback look like?

Put "what happens next" in another function.

ONE ORDER · COMMIT → NOTIFY

```
int rc = commitAsync(order, onCommitted);  
                        ^^^^^^^^^^^^^^^  
                        callback
```

issue I/O

operation completes



return rc

```
void onCommitted (CommitResult result) {  
    notify(...);  
}
```

What does a callback look like?

Put "what happens next" in another function.

ONE ORDER · COMMIT → NOTIFY

```
int rc = commitAsync(order, onCommitted);  
                        ^^^^^^^^^^^^^^^  
                        callback
```

issue I/O

operation completes



return rc

```
void onCommitted (CommitResult result) {  
    notify(...);  
}
```

A callback makes the continuation explicit as a function.

What did the call stack give us for free?

A callback lets the work outlive the stack frame.

Synchronous code

```
void activate(OrderId orderId) {  
    // Context stays in this frame.  
    auto loaded = load(orderId);  
    // The result comes back by return.  
  
    // Next is the next line.  
    compliance(orderId, loaded);  
  
}
```

Callback-based async code

```
void activate(OrderId orderId) {  
    // Context is captured for later.  
    loadAsync(orderId, [orderId](Loaded loaded) {  
  
        // Next lives in the lambda.  
        compliance(orderId, loaded);  
  
        // The result arrives through the callback.  
    });  
}
```



What did the call stack give us for free?

A callback lets the work outlive the stack frame.

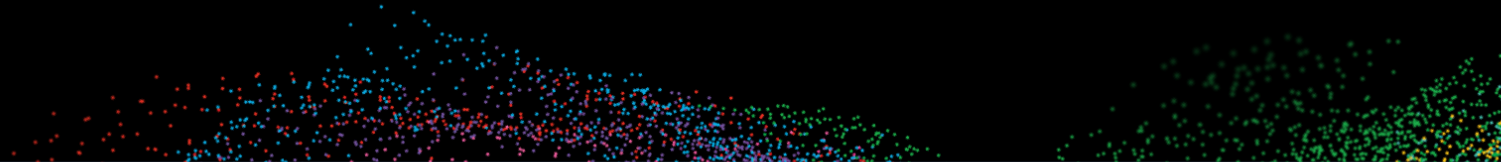
What state has to survive the wait? ·

Synchronous code

```
void activate(OrderId orderId) {  
    // Context stays in this frame.  
    auto loaded = load(orderId);  
    // The result comes back by return.  
  
    // Next is the next line.  
    compliance(orderId, loaded);  
  
}
```

Callback-based async code

```
void activate(OrderId orderId) {  
    // Context is captured for later.  
    loadAsync(orderId, [orderId](Loaded loaded) {  
  
        // Next lives in the lambda.  
        compliance(orderId, loaded);  
  
        // The result arrives through the callback.  
    });  
}
```



What did the call stack give us for free?

A callback lets the work outlive the stack frame.

What state has to survive the wait? · Where is the next step written? ·

Synchronous code

```
void activate(OrderId orderId) {  
    // Context stays in this frame.  
    auto loaded = load(orderId);  
    // The result comes back by return.  
  
    // Next is the next line.  
    compliance(orderId, loaded);  
  
}
```

Callback-based async code

```
void activate(OrderId orderId) {  
    // Context is captured for later.  
    loadAsync(orderId, [orderId](Loaded loaded) {  
  
        // Next lives in the lambda.  
        compliance(orderId, loaded);  
  
        // The result arrives through the callback.  
    });  
}
```



What did the call stack give us for free?

A callback lets the work outlive the stack frame.

What state has to survive the wait? · Where is the next step written? · How does the result arrive?

Synchronous code

```
void activate(OrderId orderId) {  
    // Context stays in this frame.  
    auto loaded = load(orderId);  
    // The result comes back by return.  
  
    // Next is the next line.  
    compliance(orderId, loaded);  
  
}
```

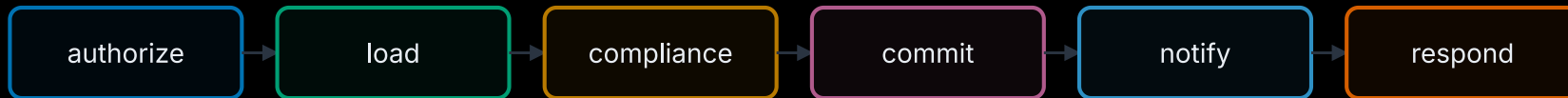
Callback-based async code

```
void activate(OrderId orderId) {  
    // Context is captured for later.  
    loadAsync(orderId, [orderId](Loaded loaded) {  
  
        // Next lives in the lambda.  
        compliance(orderId, loaded);  
  
        // The result arrives through the callback.  
    });  
}
```



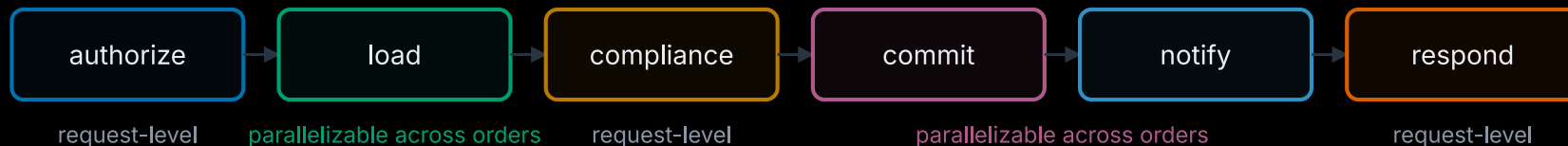
Challenge: Scale it to 2,000 orders

The six steps stay the same. Their execution shapes do not.



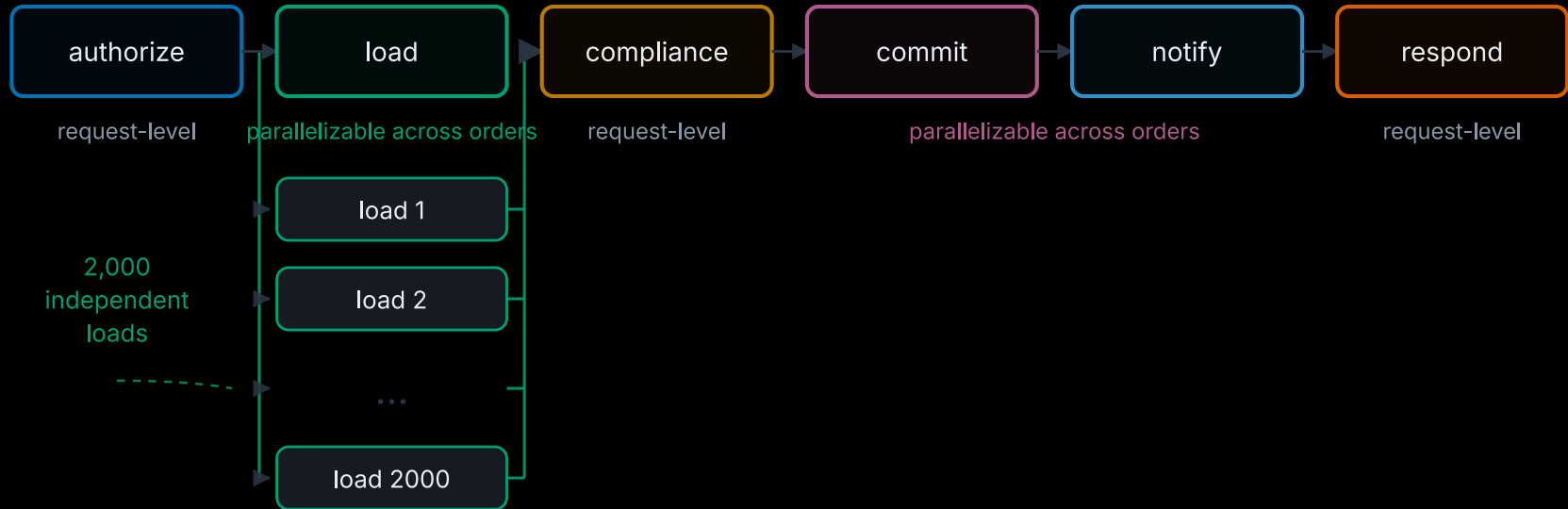
Challenge: Scale it to 2,000 orders

The six steps stay the same. Their execution shapes do not.



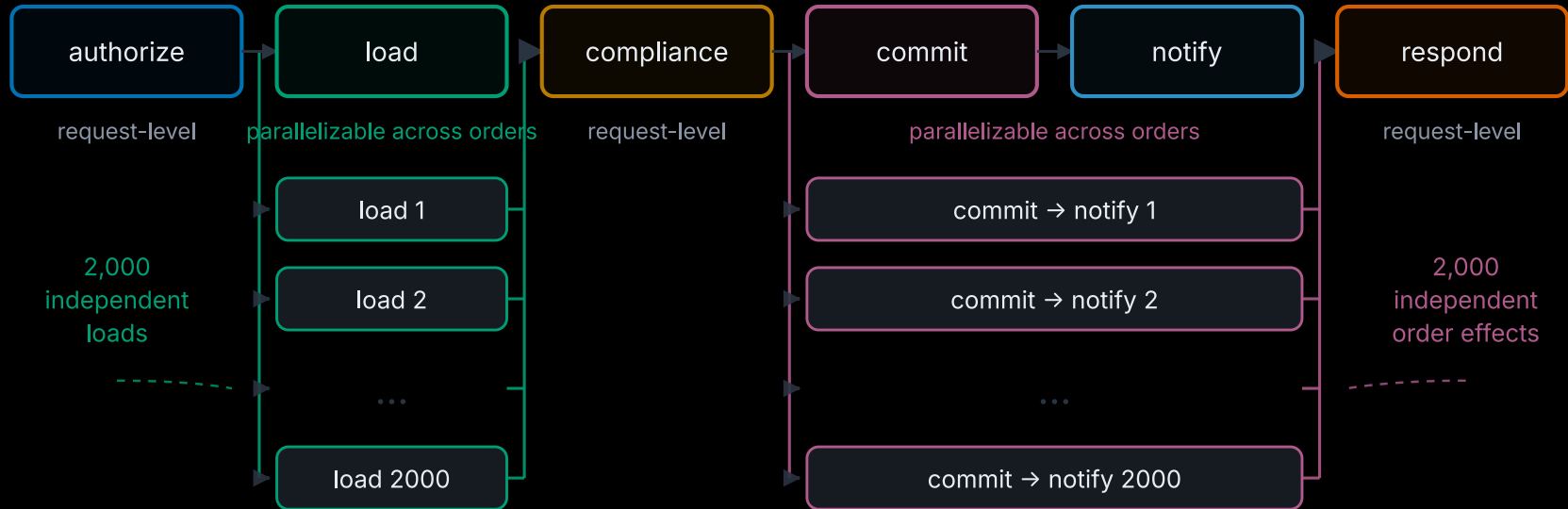
Challenge: Scale it to 2,000 orders

The six steps stay the same. Their execution shapes do not.



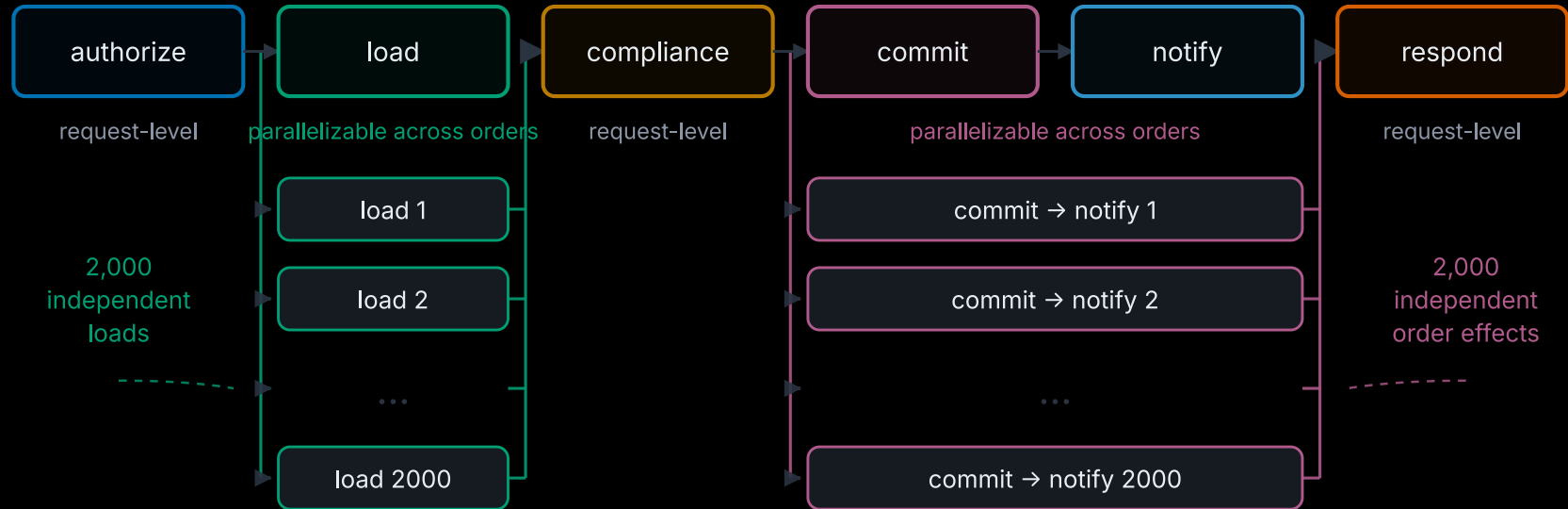
Challenge: Scale it to 2,000 orders

The six steps stay the same. Their execution shapes do not.



Challenge: Scale it to 2,000 orders

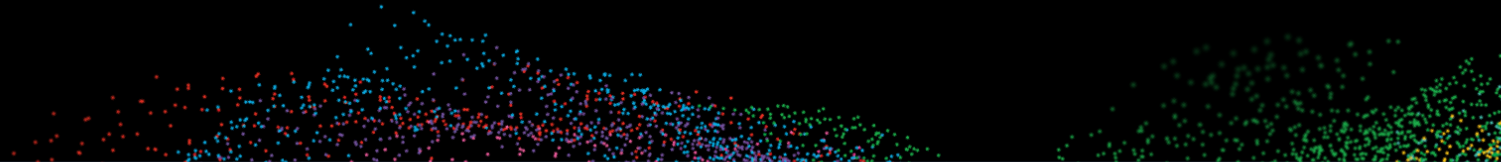
The six steps stay the same. Their execution shapes do not.



The business workflow stayed simple. The orchestration did not.

What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.



What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
commitAsync(..., onCommitted)
```



What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
commitAsync(..., onCommitted)
```



```
commit_handler.cpp  
onCommitted(result)  
sendNotify(..., onNotified)
```



What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
commitAsync(..., onCommitted)
```



```
commit_handler.cpp  
onCommitted(result)  
sendNotify(..., onNotified)
```



```
notify_handler.cpp  
onNotified(result)  
recordOutcome(...)
```



What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
commitAsync(..., onCommitted)
```



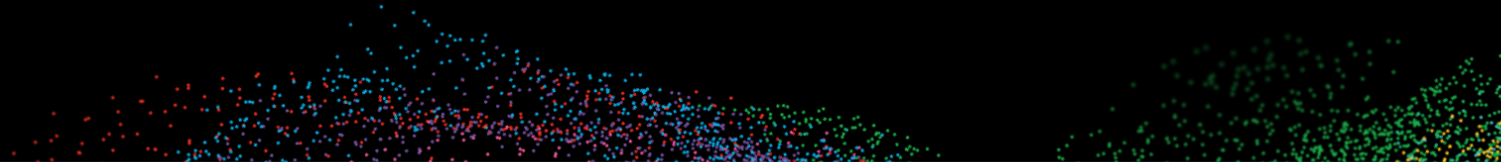
```
commit_handler.cpp  
onCommitted(result)  
sendNotify(..., onNotified)
```



```
notify_handler.cpp  
onNotified(result)  
recordOutcome(...)
```



```
outcome_tracker.cpp  
recordOutcome(outcome)  
- one order, done
```



What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
  commitAsync(..., onCommitted)
```

```
commit_handler.cpp  
onCommitted(result)  
  sendNotify(..., onNotified)
```

```
notify_handler.cpp  
onNotified(result)  
  recordOutcome(...)
```

```
outcome_tracker.cpp  
recordOutcome(outcome)  
  — one order, done
```

LOCAL VIEW

Each handler is easy to reason about.

The next continuation is explicit, and each function can stay small.

What does a callback-based workflow look like?

Complex callbacks are usually refactored into their own components.

```
order_effects.cpp  
startCommit(order)  
commitAsync(..., onCommitted)
```

```
commit_handler.cpp  
onCommitted(result)  
sendNotify(..., onNotified)
```

```
notify_handler.cpp  
onNotified(result)  
recordOutcome(...)
```

```
outcome_tracker.cpp  
recordOutcome(outcome)  
- one order, done
```

LOCAL VIEW

Each handler is easy to reason about.

The next continuation is explicit, and each function can stay small.

WORKFLOW VIEW

The sequence is distributed.

Where is this written?

Nowhere in one place.

The callbacks might be locally readable, but the workflow is scattered around.

Workflow anatomy · Callback-based

Where do control flow, data flow, and completion live now?

	Sync	Callback-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch



Workflow anatomy · Callback-based

Where do control flow, data flow, and completion live now?

	Sync	Callback-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch

Callbacks make “next” explicit, but completion and data move into bookkeeping.



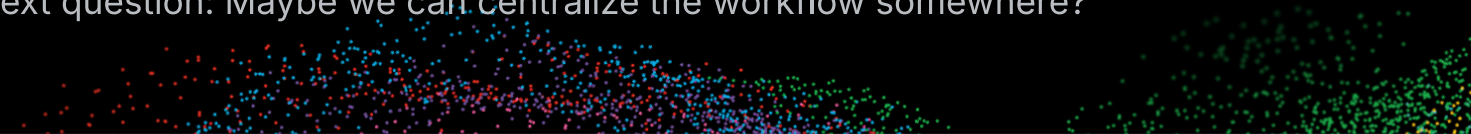
Workflow anatomy · Callback-based

Where do control flow, data flow, and completion live now?

	Sync	Callback-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch

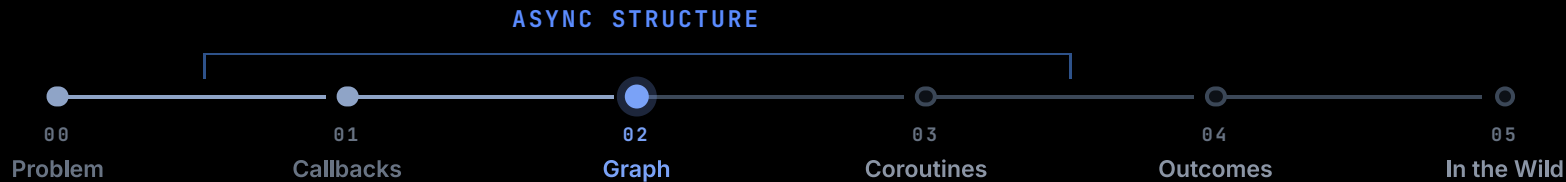
Callbacks make “next” explicit, but completion and data move into bookkeeping.

Next question: Maybe we can centralize the workflow somewhere?

A decorative particle effect at the bottom of the slide, consisting of a dense cluster of small, multi-colored dots (red, green, blue, and yellow) that appear to be floating or settling, creating a sense of motion and complexity.

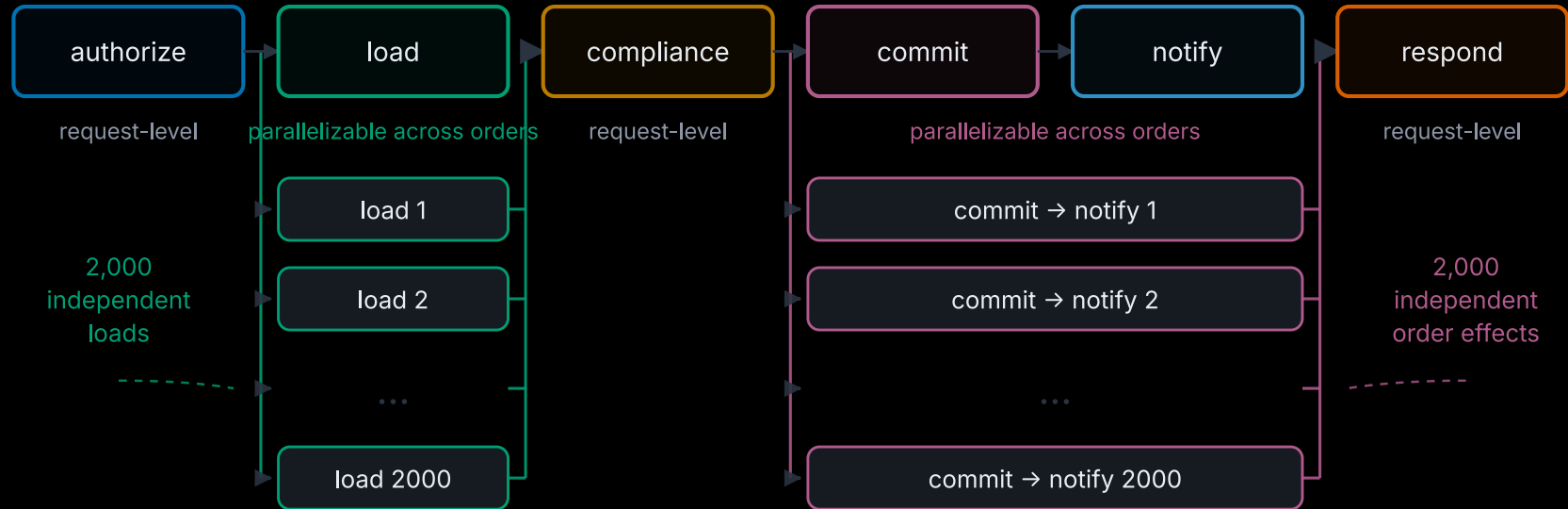
Part 2 · Graph

What if we declare the dependencies instead of tracing continuations?



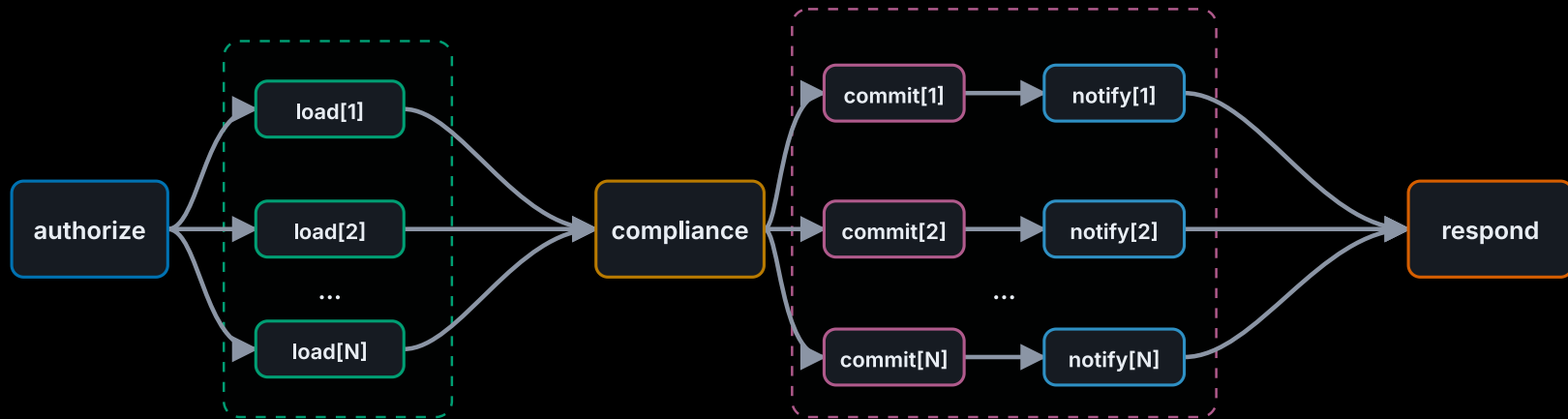
Model the workflow explicitly

A common approach: Represent the workflow as a task graph



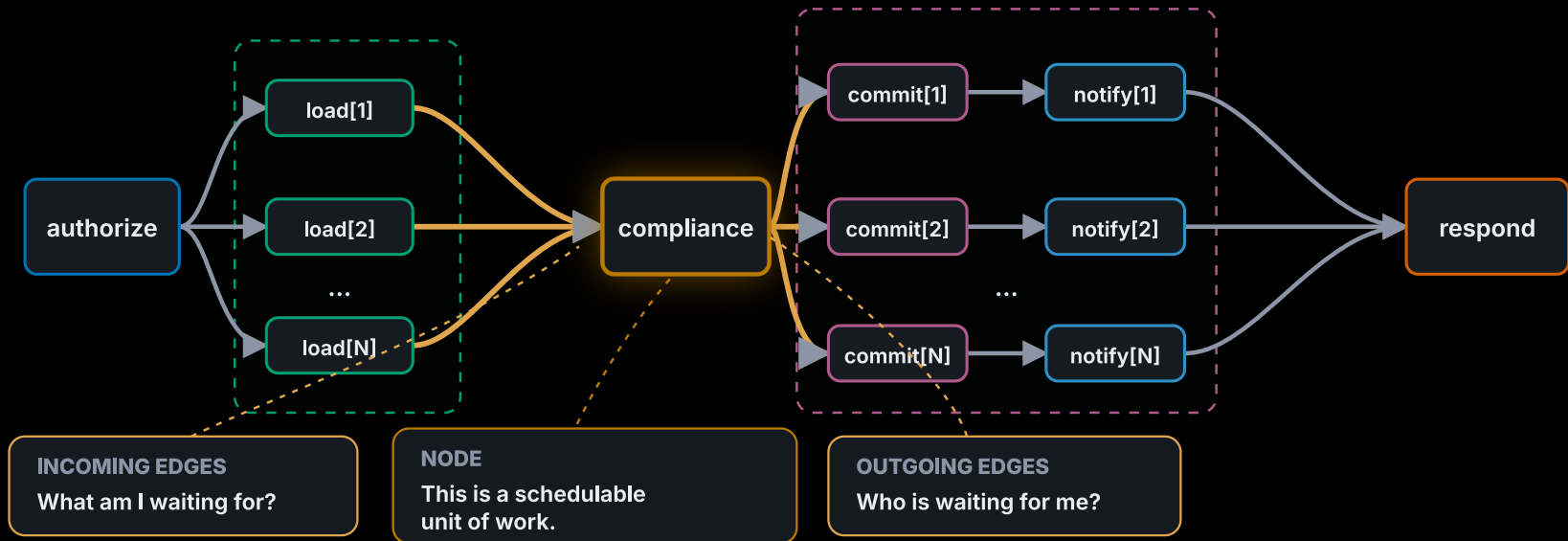
The workflow becomes a graph

Why does the graph actually improve orchestration?



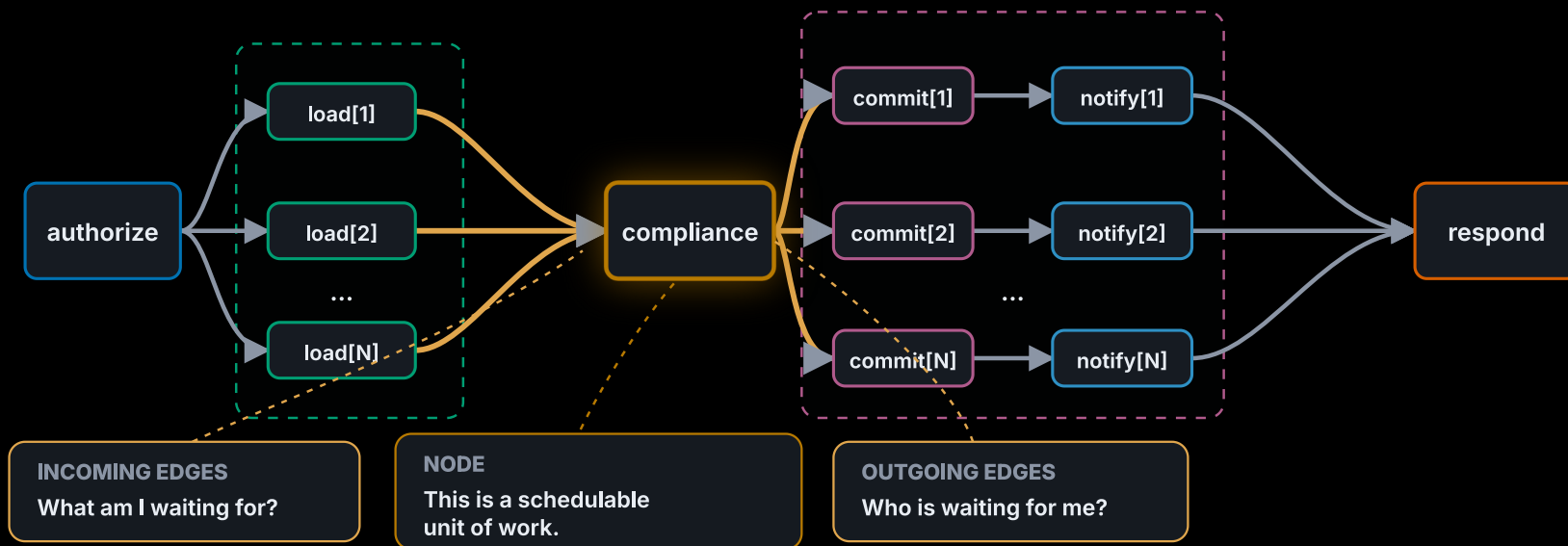
The workflow becomes a graph

Why does the graph actually improve orchestration?



The workflow becomes a graph

Why does the graph actually improve orchestration?



The **runtime** tracks dependencies and makes tasks ready when they are satisfied.

The workflow is no longer a trace. It is a **model the runtime can execute**.

The graph, declared in code

/ Illustrative API / The code is another representation of the same picture.

```
Flow buildActivationFlow()
{
    Flow flow;
    flow.emplace("Authorize",    &Nodes::authorize);
    flow.emplace("LoadFanOut",  &Nodes::load);
    flow.emplace("Compliance",  &Nodes::compliance);
    flow.emplace("WriteFanOut", &Nodes::writeAndNotify);
    flow.emplace("Respond",     &Nodes::respond);

    flow.addConnection("Authorize",    "LoadFanOut");
    flow.addConnection("LoadFanOut",   "Compliance");
    flow.addConnection("Compliance",   "WriteFanOut");
    flow.addConnection("WriteFanOut",  "Respond");

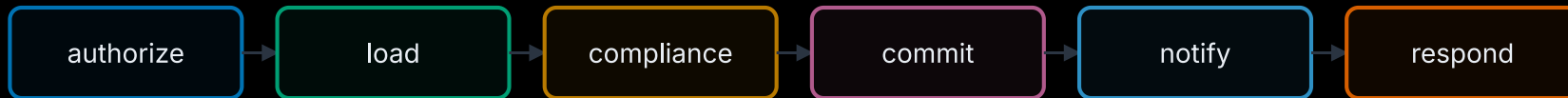
    return flow;
}
```

`emplace` tells the runtime what code this node runs `addConnectio` tells it when that node may run

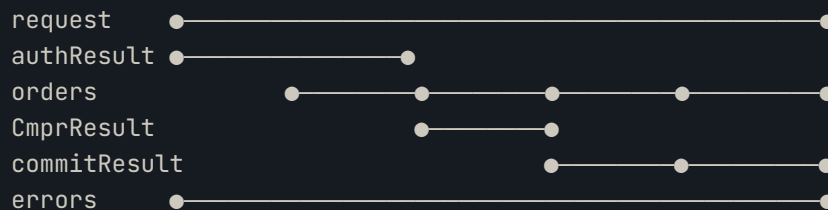


But the data has a different shape

When control and data take different paths, that mismatch has to be represented somewhere.

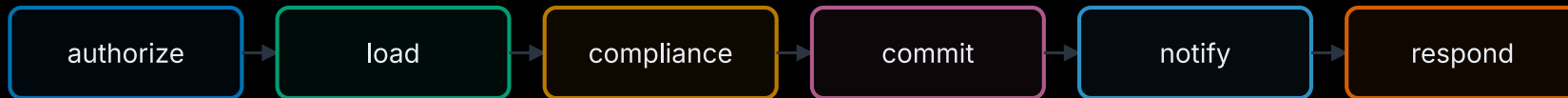


DATA LIFETIMES

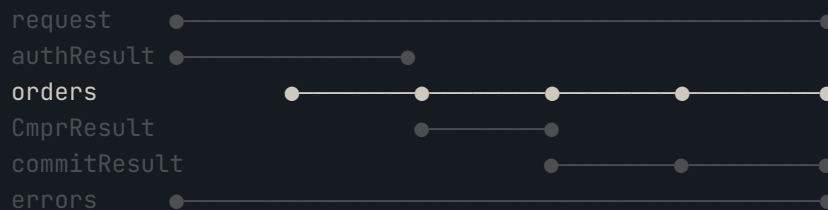


But the data has a different shape

When control and data take different paths, that mismatch has to be represented somewhere.

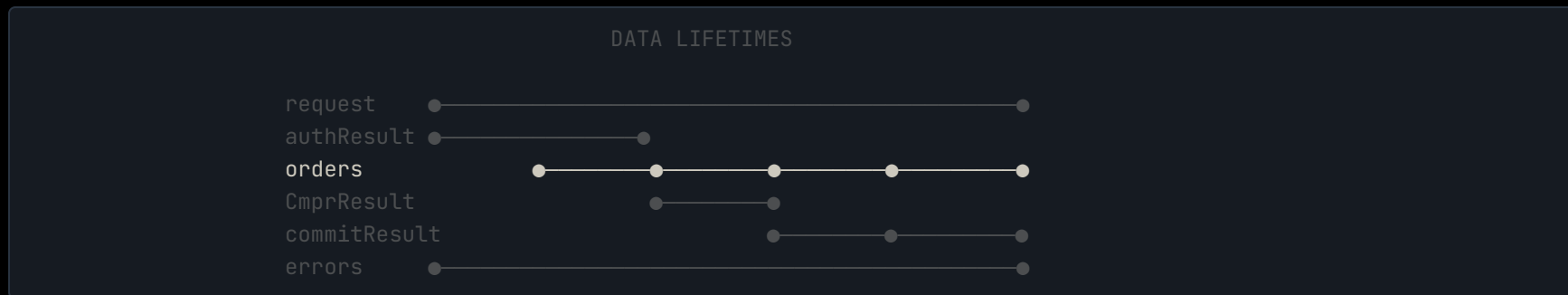
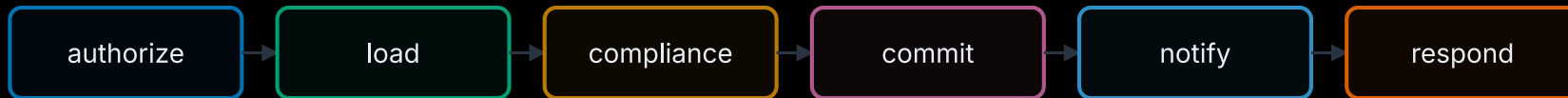


DATA LIFETIMES



But the data has a different shape

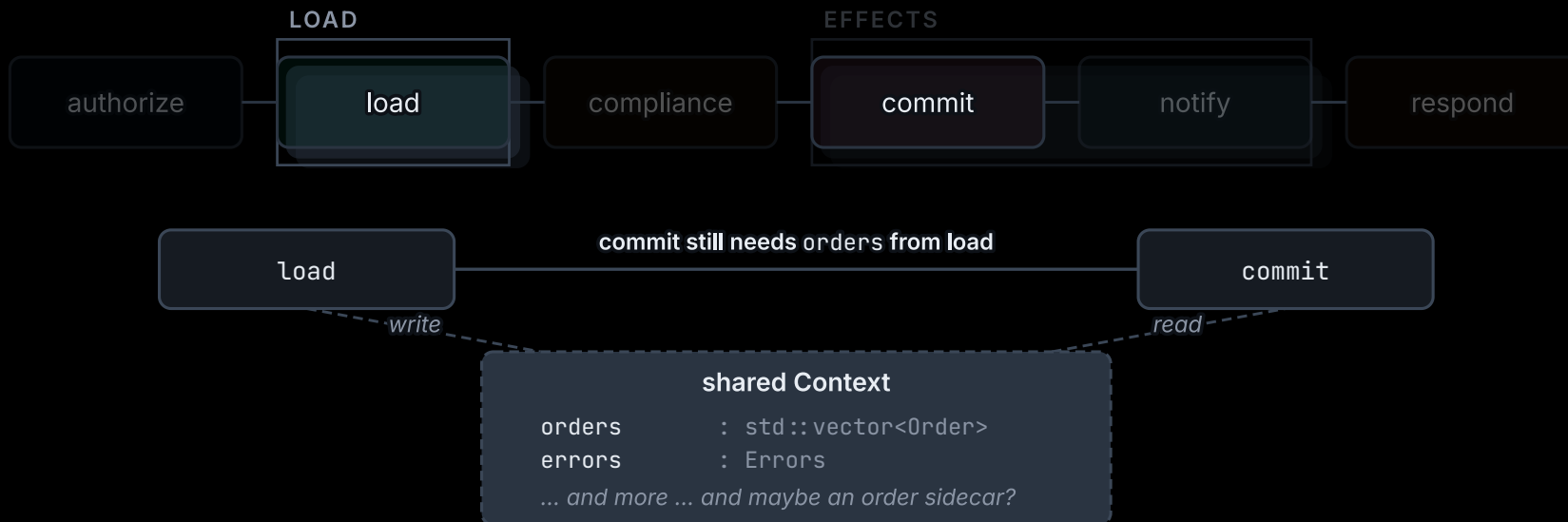
When control and data take different paths, that mismatch has to be represented somewhere.



Cross-step state has to live somewhere.

But the data has a different shape

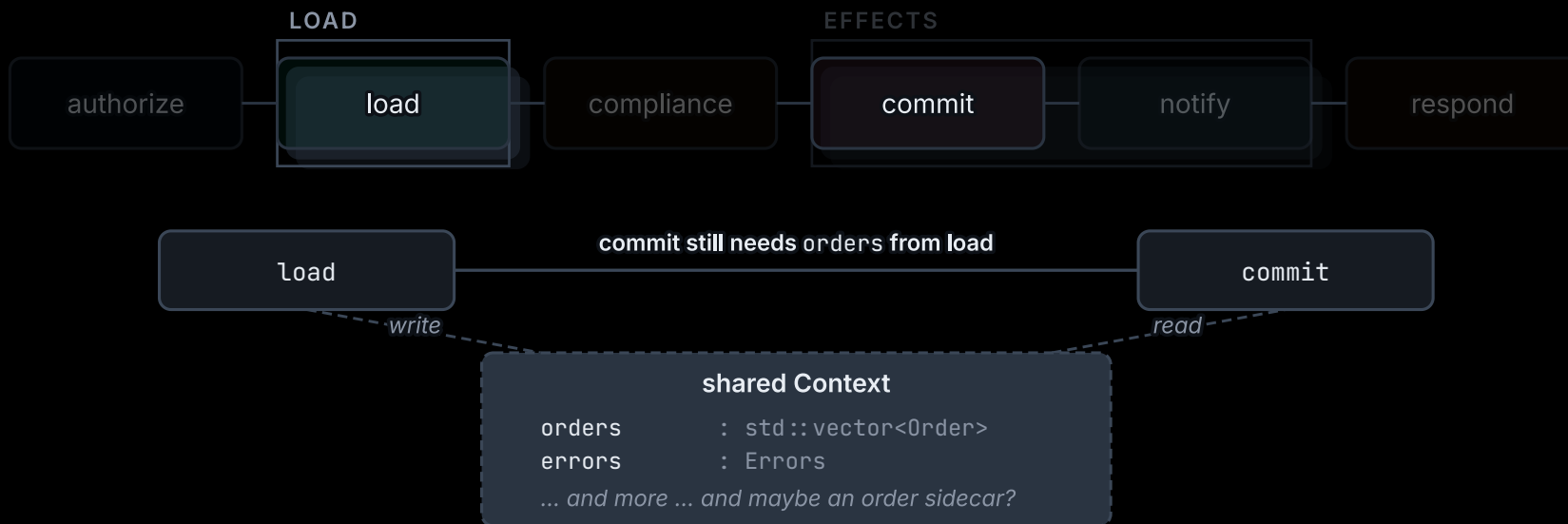
When control and data take different paths, that mismatch has to be represented somewhere.



In this design, that bridge is the shared Context.

But the data has a different shape

When control and data take different paths, that mismatch has to be represented somewhere.



In this design, that bridge is the shared Context.

Cross-step values accumulate in the same shared interface.

Workflow anatomy · Graph-based

	Sync	Callback-based	Graph-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain	declared edges
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state	shared <code>Context</code>
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch	runtime-managed join



Workflow anatomy · Graph-based

	Sync	Callback-based	Graph-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain	declared edges
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state	shared <code>Context</code>
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch	runtime-managed join

The graph made control dependencies explicit and let the runtime handle joins.
But our data still flowed through a shared `Context`



Workflow anatomy · Graph-based

	Sync	Callback-based	Graph-based
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain	declared edges
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state	shared <code>Context</code>
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch	runtime-managed join

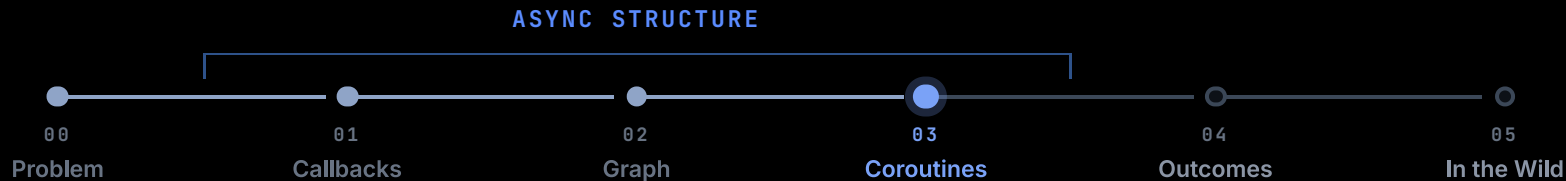
The graph made control dependencies explicit and let the runtime handle joins.
But our data still flowed through a shared `Context`

Next question: Can control flow and data flow meet again in ordinary code?



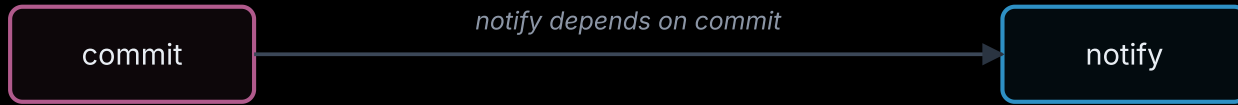
Part 3 · Coroutines

Can control and data live together again?



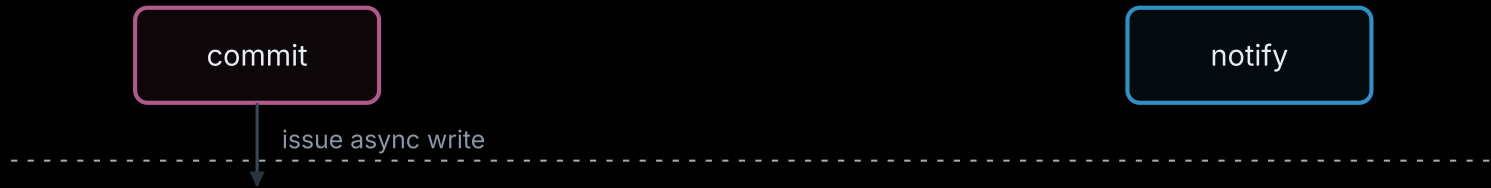
Continuation from first principles

A continuation preserves where to resume and what must survive.



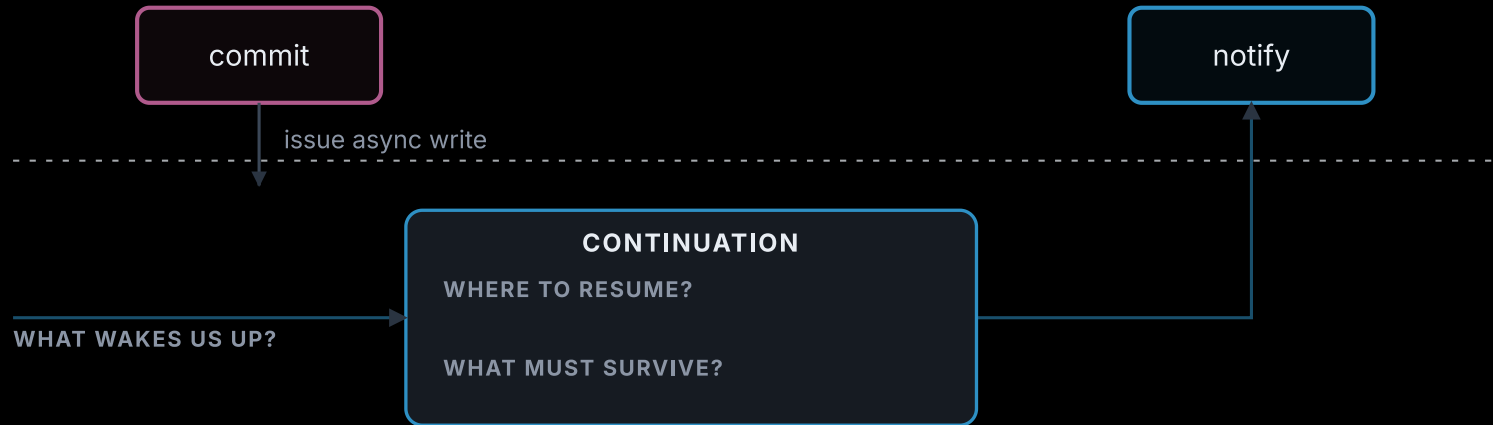
Continuation from first principles

A continuation preserves where to resume and what must survive.



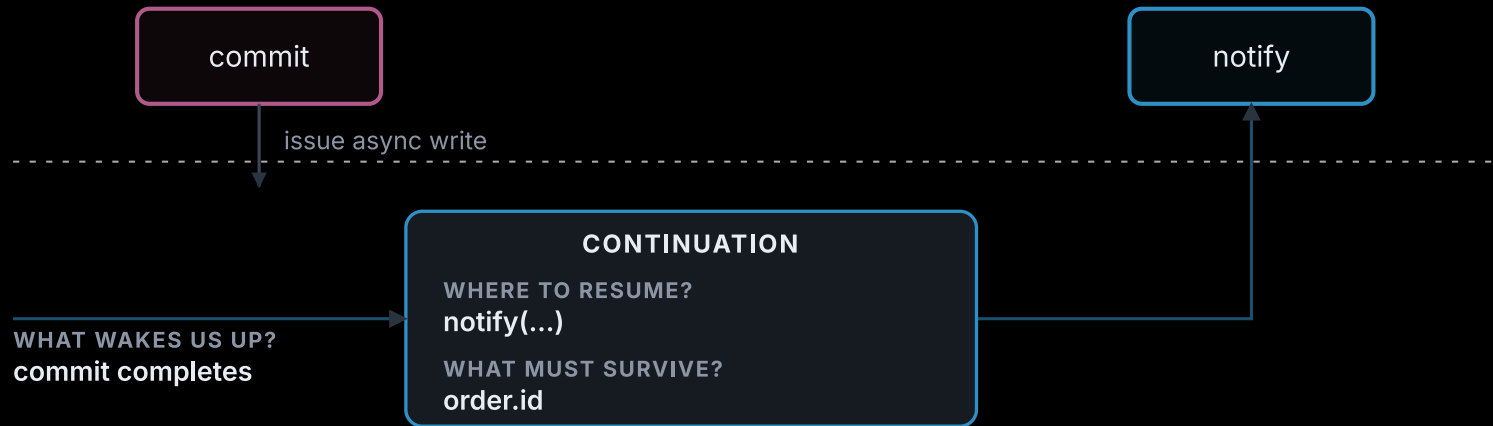
Continuation from first principles

A continuation preserves where to resume and what must survive.



Continuation from first principles

A continuation preserves where to resume and what must survive.



How does `co_await` connect the continuation?

```
auto result = co_await commit(order);  
notify(orderId, result);
```

`co_await commit(order)`

`notify(orderId, result);`

How does `co_await` connect the continuation?

```
auto result = co_await commit(order);  
notify(orderId, result);
```

`co_await commit(order)`

`notify(orderId, result);`

① CONNECT + SUSPEND

connect completion
to coroutine

LIBRARY



start / register
async operation

LIBRARY

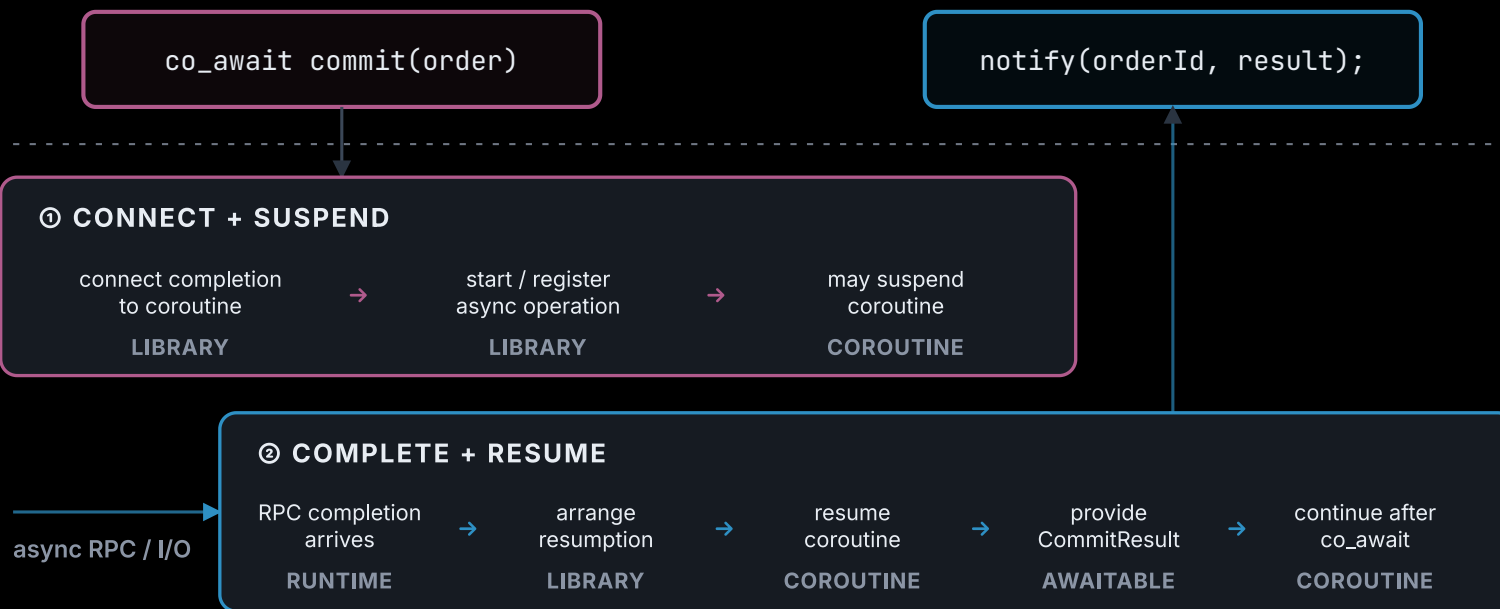


may suspend
coroutine

COROUTINE

How does `co_await` connect the continuation?

```
auto result = co_await commit(order);  
notify(orderId, result);
```



What does `co_await` give us back?

The state machine still exists — but we no longer have to write it.

```
auto orderId = order.id;  
  
auto result = co_await commit(order);  
  
notify(orderId, result);
```



What does `co_await` give us back?

The state machine still exists — but we no longer have to write it.

```
auto orderId = order.id;  
  
auto result = co_await commit(order);  
  
notify(orderId, result);
```

STATE — Locals still look local

Live locals survive the suspension.



What does `co_await` give us back?

The state machine still exists — but we no longer have to write it.

```
auto orderId = order.id;  
  
auto result = co_await commit(order);  
  
notify(orderId, result);
```

STATE — Locals still look local

Live locals survive the suspension.

NEXT — The next line is the continuation

No handler to find.



What does `co_await` give us back?

The state machine still exists — but we no longer have to write it.

```
auto orderId = order.id;  
  
auto result = co_await commit(order);  
  
notify(orderId, result);
```

STATE — Locals still look local

Live locals survive the suspension.

NEXT — The next line is the continuation

No handler to find.

RESULT — `co_await` yields the async result

The result flows into an ordinary local variable.



What does `co_await` give us back?

The state machine still exists — but we no longer have to write it.

```
auto orderId = order.id;  
  
auto result = co_await commit(order);  
  
notify(orderId, result);
```

STATE — Locals still look local

Live locals survive the suspension.

NEXT — The next line is the continuation

No handler to find.

RESULT — `co_await` yields the async result

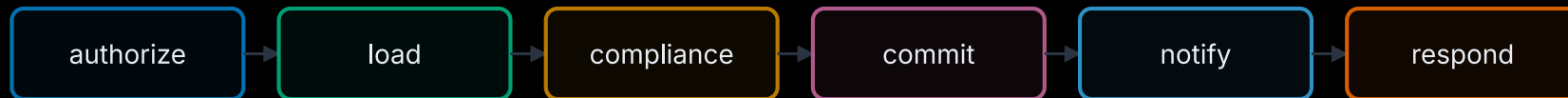
The result flows into an ordinary local variable.

One asynchronous path reads normally again

Coroutines make the continuation a language feature instead of hand-written plumbing.



The workflow, again



```
Task<Response> activateOrders(Request request)
{
    co_await authorize(request);

    auto orders = co_await loadOrders(request.orderIds);
    auto checked = co_await checkCompliance(std::move(orders));

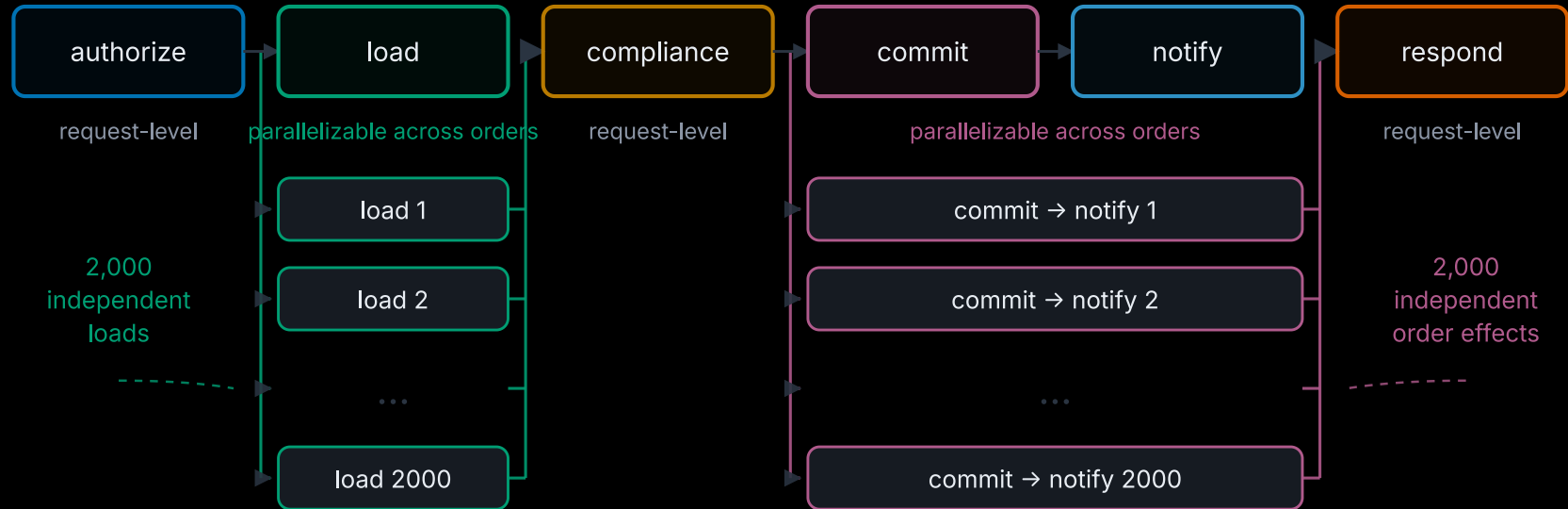
    auto committed = co_await commit(std::move(checked));
    co_await notifyDownstream(committed);

    co_return makeResponse(std::move(committed), {});
}
```

Business ordering matches source-code ordering — again!

One workflow, many independent operations

One path is readable. What about its children?



The parent owns the fan-out. The parent does not finish before the scope does.

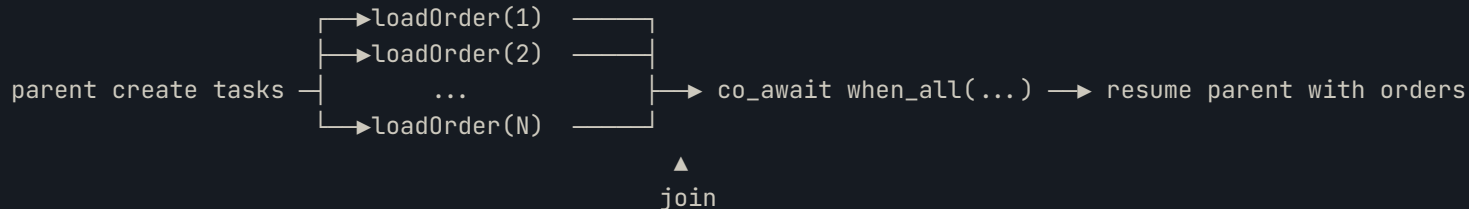
What is Task<T>?

```
Task<Order> loadOrder(OrderId orderId);  
  
Order order = co_await loadOrder(orderId);
```

- Task<T> is a library-level abstraction representing one async operation.
 - Its normal completion produces a T.
- Task<T> is awaitable.
- co_await says: I depend on this result.
 - When the result is ready, co_await loadOrder(...) produces an Order.

when_all for structured concurrency

Compose child operations, suspend the parent at the join, and resume when all have completed.



```
std::vector<Task<Order>> tasks;

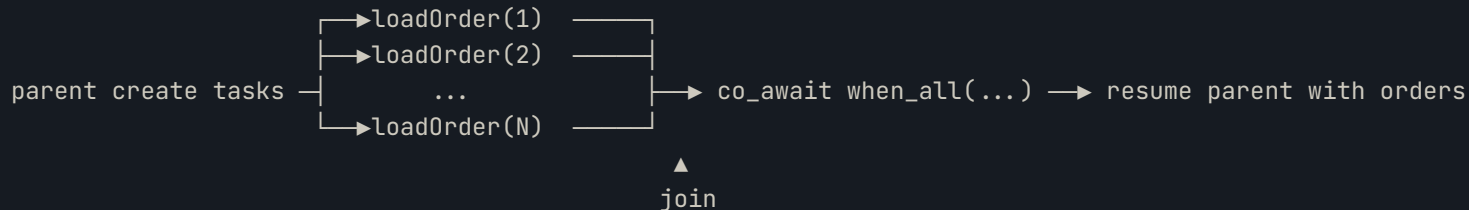
for (auto id : orderIds)
    tasks.push_back(loadOrder(id));

auto orders =
    co_await when_all(std::move(tasks));

process(orders);
```

when_all for structured concurrency

Compose child operations, suspend the parent at the join, and resume when all have completed.



```
std::vector<Task<Order>> tasks;

for (auto id : orderIds)
    tasks.push_back(loadOrder(id));

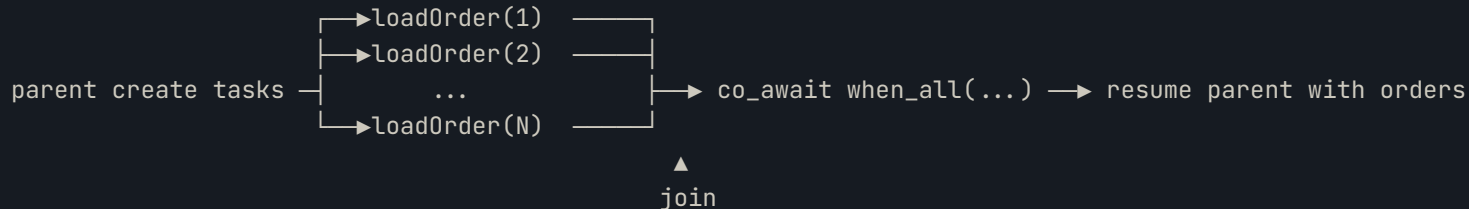
auto orders =
    co_await when_all(std::move(tasks));

process(orders);
```

- **fan out:** Many independent child operations

when_all for structured concurrency

Compose child operations, suspend the parent at the join, and resume when all have completed.



```
std::vector<Task<Order>> tasks;

for (auto id : orderIds)
    tasks.push_back(loadOrder(id));

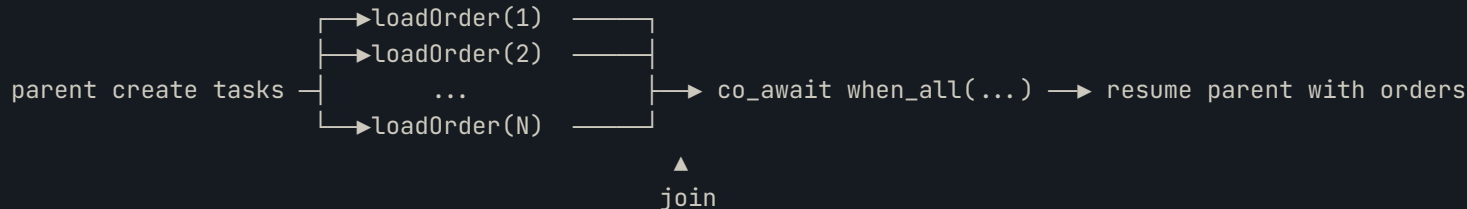
auto orders =
    co_await when_all(std::move(tasks));

process(orders);
```

- **fan out:** Many independent child operations
- **join:** The parent suspends until every child completes. parent suspended $\neq$ thread blocked.

when_all for structured concurrency

Compose child operations, suspend the parent at the join, and resume when all have completed.



```
std::vector<Task<Order>> tasks;

for (auto id : orderIds)
    tasks.push_back(loadOrder(id));

auto orders =
    co_await when_all(std::move(tasks));

process(orders);
```

- **fan out:** Many independent child operations
- **join:** The parent suspends until every child completes. parent suspended $\neq$ thread blocked.
- **resume:** All child results are local again

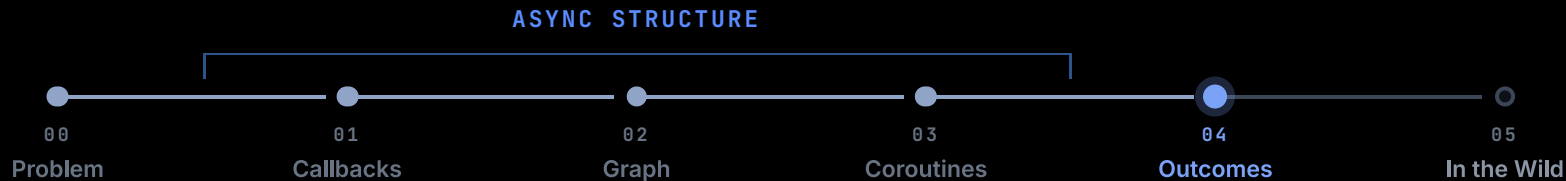
Workflow anatomy · Coroutines

	Sync	Callback-based	Graph-based	Coroutines
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain	declared edges	top-to-bottom code
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state	shared <code>Context</code>	locals / <code>co_await</code> results
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch	runtime-managed join	<code>co_await</code> / <code>when_all</code>



Part 4 · Outcomes

Completion tells us when. How do we represent what happened?



So far, every operation succeeded.

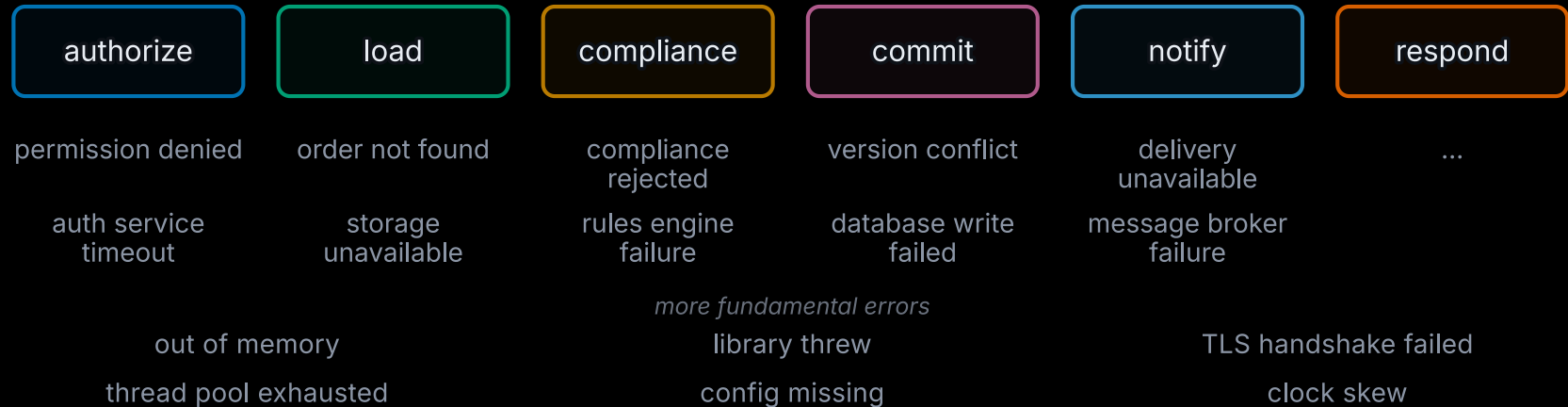
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

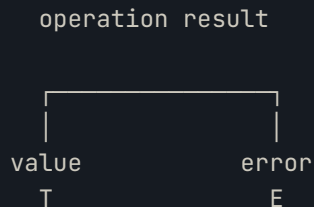
Success is not the only outcome



Right now, business outcomes and exceptional failures collapse into the same path.

Can the outcome flow stay as readable as the control flow?

What does failure mean for the result?



Once failure is part of the contract, the result is a sum type: $T \mid E$.

At this boundary, the result is in one state: T or E.

Exactly what belongs in T vs. E is part of the API contract.

Why not the mechanisms we already have?

We need one logical result, with both alternatives visible in the type.

Status code + out-param

sync world

```
Status loadOrder(OrderId, Order& out);
```

async world

```
Task<Status> loadOrder(OrderId id, Order& out);
```

One logical result, two output channels.

result — { Status return channel
 — Order& out-parameter

Explicit — but split across two channels.

Exception

Clean happy path

```
Task<Order> loadOrder(OrderId id);
```

```
auto order = co_await loadOrder(id);
```

```
OrderNotFound?  
ServiceUnavailable?
```

One logical result, two control paths,

result — { Order normal return
 — OrderNotFoundException exceptional path

Clean — but the alternative is absent from the type.

`std::expected<T, E>`: One result, two states

There are two legitimate outcomes: either I have a T, or I have an E.



`std::expected<T, E>`: One result, two states

There are two legitimate outcomes: either I have a T, or I have an E.

`std::expected<OrderT, ErrorE>`

^T PROMISED VALUE

Order

the operation produced the value it promised

^E ALTERNATIVE OUTCOME

Error

user-defined type · explains why there is no Order

Exactly one state is present.

`std::expected<T, E>`: One result, two states

There are two legitimate outcomes: either I have a T, or I have an E.

`std::expected<OrderT, ErrorE>`

^T PROMISED VALUE

Order

the operation produced the value it promised

^E ALTERNATIVE OUTCOME

Error

user-defined type · explains why there is no Order

Exactly one state is present.

How to use

```
if (result) {  
    use(*result);           // T  
} else {  
    handle(result.error()); // E  
}
```

Returning the E state

```
// Example: enum class Error { notFound };  
  
return std::unexpected(  
    Error::notFound  
);
```

`Task<std::expected<T, E>>`: Two questions, two axes

One async logical result, with both alternatives.



Task<std::expected<T, E>>: Two questions, two axes

One async logical result, with both alternatives.



Task<std::expected<T, E>>: Two questions, two axes

One async logical result, with both alternatives.

COMPLETION
when is it done?



`Task<std::expected<T, E>>`: Two questions, two axes

One async logical result, with both alternatives.

COMPLETION
when is it done?

		OUTCOME what did it produce?	
		the value	the value or why not
direct		<code>T</code>	<code>std::expected<T, E></code>
asynchronous		<code>Task<T></code>	

`Task<std::expected<T, E>>`: Two questions, two axes

One async logical result, with both alternatives.

COMPLETION
when is it done?

		OUTCOME what did it produce?	
		the value	the value or why not
direct		<code>T</code>	<code>std::expected<T, E></code>
asynchronous		<code>Task<T></code>	<code>Task<std::expected<T, E>></code>

Task<std::expected<T, E>>: Two questions, two axes

One async logical result, with both alternatives.

OUTCOME

what did it produce?

the value

the value or why not

COMPLETION
when is it done?

direct

T

std::expected<T, E>

asynchronous

Task<T>

Task<std::expected<T, E>>

Task<...> answers when we continue.

std::expected<T, E> answers what we continue with.

Control flow and outcome flow are local again

```
Task<std::expected<CommitResult, Error>> activateOrder(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return std::unexpected(authorized.error());

    auto order = co_await loadOrder(request.orderId);
    if (!order) co_return std::unexpected(order.error());

    auto checked = co_await checkCompliance(*order);
    if (!checked) co_return std::unexpected(checked.error());

    auto committed = co_await commit(*checked);
    if (!committed) co_return std::unexpected(committed.error());

    auto notified = co_await notify(*committed);
    if (!notified) co_return std::unexpected(notified.error());

    co_return *committed;
}
```

Control flow and outcome flow are local again

```
Task<std::expected<CommitResult, Error>> activateOrder(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return std::unexpected(authorized.error());

    auto order = co_await loadOrder(request.orderId);
    if (!order) co_return std::unexpected(order.error());

    auto checked = co_await checkCompliance(*order);
    if (!checked) co_return std::unexpected(checked.error());

    auto committed = co_await commit(*checked);
    if (!committed) co_return std::unexpected(committed.error());

    auto notified = co_await notify(*committed);
    if (!notified) co_return std::unexpected(notified.error());

    co_return *committed;
}
```

Control flow and outcome flow are local again

```
Task<std::expected<CommitResult, Error>> activateOrder(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return std::unexpected(authorized.error());

    auto order = co_await loadOrder(request.orderId);
    if (!order) co_return std::unexpected(order.error());

    auto checked = co_await checkCompliance(*order);
    if (!checked) co_return std::unexpected(checked.error());

    auto committed = co_await commit(*checked);
    if (!committed) co_return std::unexpected(committed.error());

    auto notified = co_await notify(*committed);
    if (!notified) co_return std::unexpected(notified.error());

    co_return *committed;
}
```

Part 5 · In the Wild

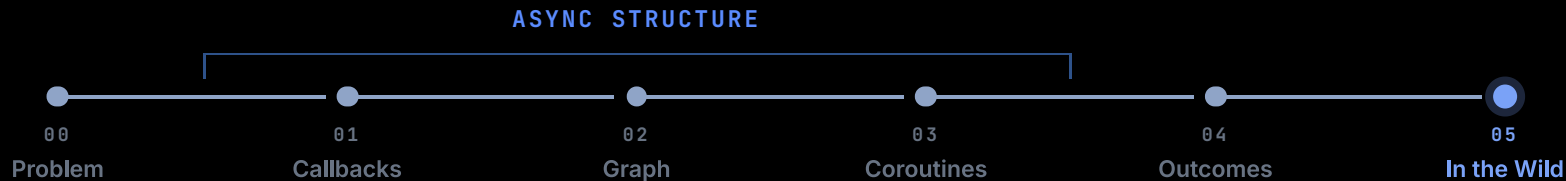
Can a policy change stay local?

Execution policy – how the work runs

batch the work

Outcome policy – what counts as success

partial success / partial failure



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

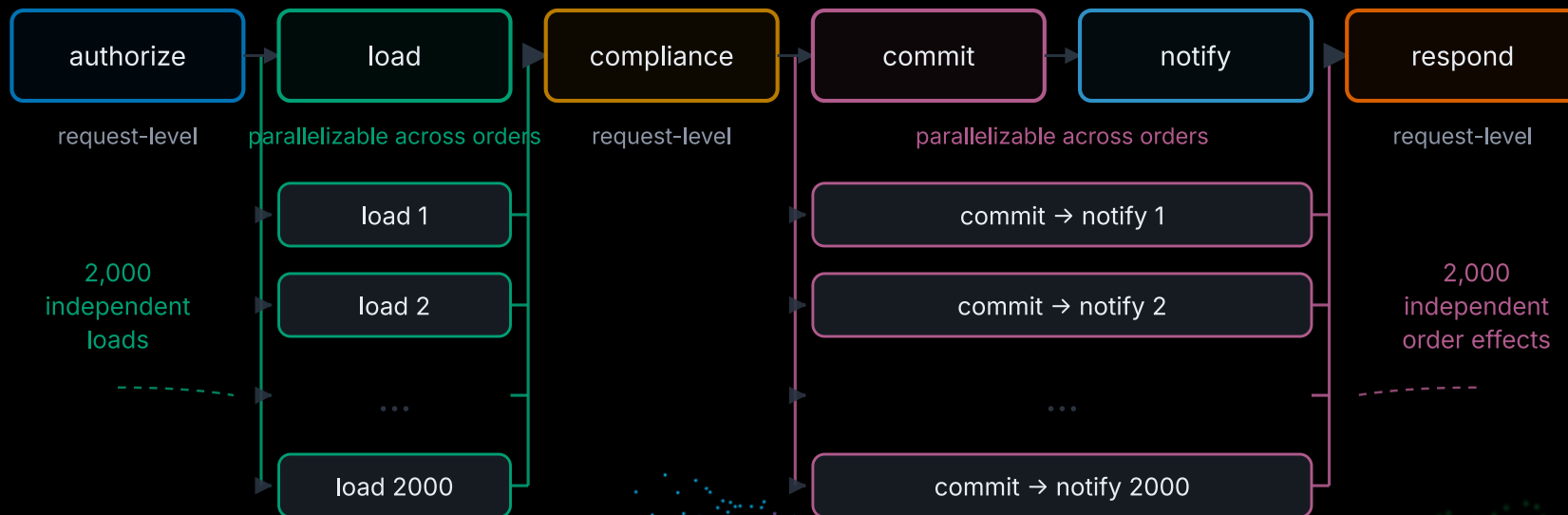
Bloomberg

Engineering

Execution policy: Batch the work

Same business dependency. A different async topology inside loadOrders.

The workflow shape stays stable while the execution shape evolves.



Execution policy: Batch the work

Same business dependency. A different async topology inside loadOrders.

The workflow shape stays stable while the execution shape evolves.



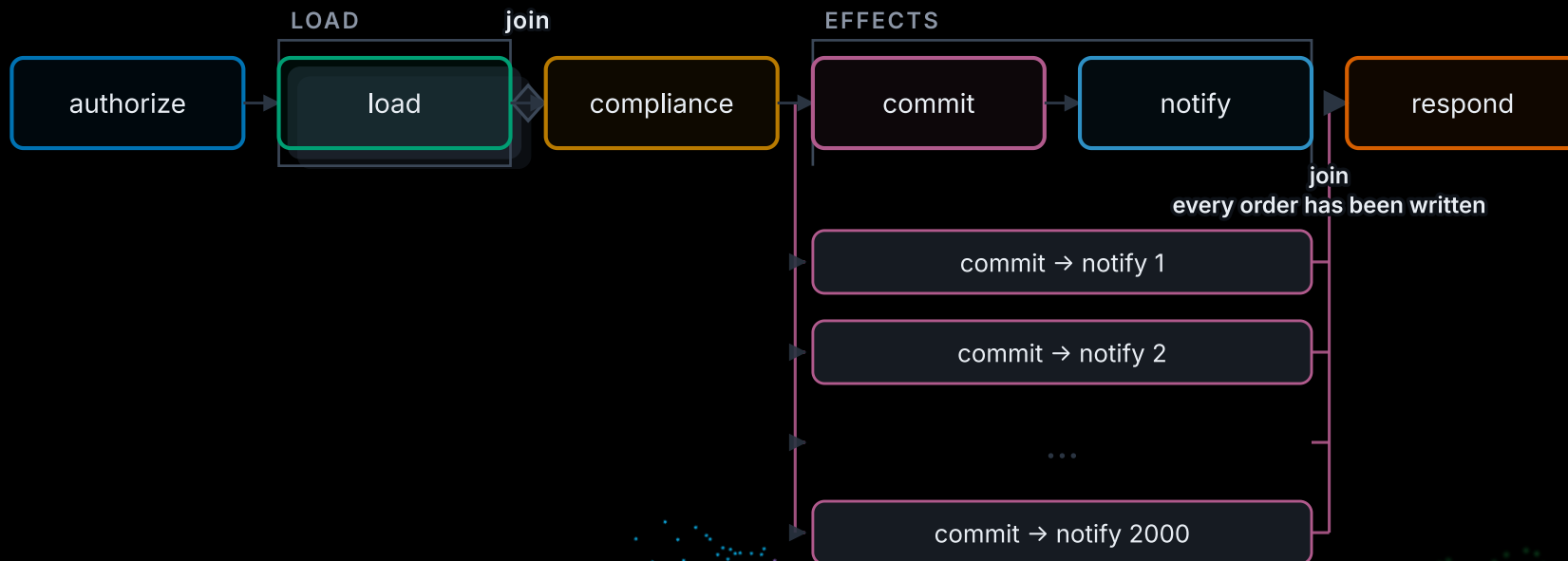
each child returns Batch

LOAD → the parent cannot return until every child is done

Execution policy: Batch the work

Same business dependency. A different async topology inside loadOrders.

The workflow shape stays stable while the execution shape evolves.



each child returns `CommitResult`

EFFECTS — the parent cannot return until every child is done

Batching as a local policy

One workflow step can own its own async structure.

Outer Workflow

```
auto orders = co_await loadOrders(orderIds);  
auto check  = co_await checkCompliance(orders);
```

per-order fan-out

```
Task<vector<Order>>  
loadOrders(span<OrderId> ids)  
{  
    vector<Task<Order>> tasks;  
  
    for (auto id : ids)  
        tasks.push_back(loadOrder(id));  
  
    co_return co_await when_all(tasks);  
}
```

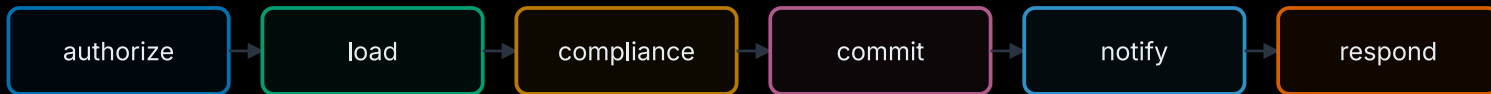
batched fan-out

```
Task<vector<Order>>  
loadOrders(span<OrderId> ids)  
{  
    vector<Task<vector<Order>>> tasks;  
  
    for (auto batch : into_batches(ids, 100))  
        tasks.push_back(loadBatch(batch));  
  
    co_return flatten(  
        co_await when_all(tasks));  
}
```

The workflow reads the same when its execution shape changes.

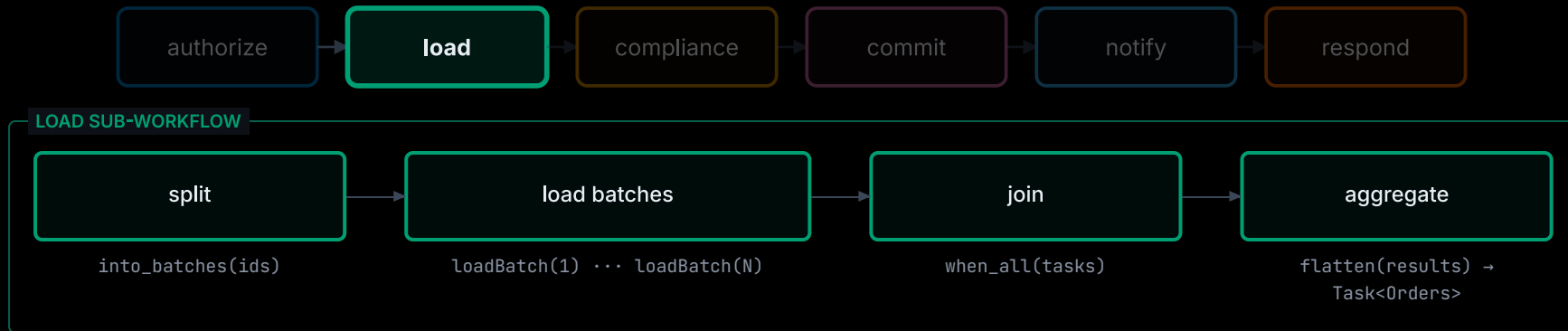
Async composition preserves structure

REQUEST WORKFLOW



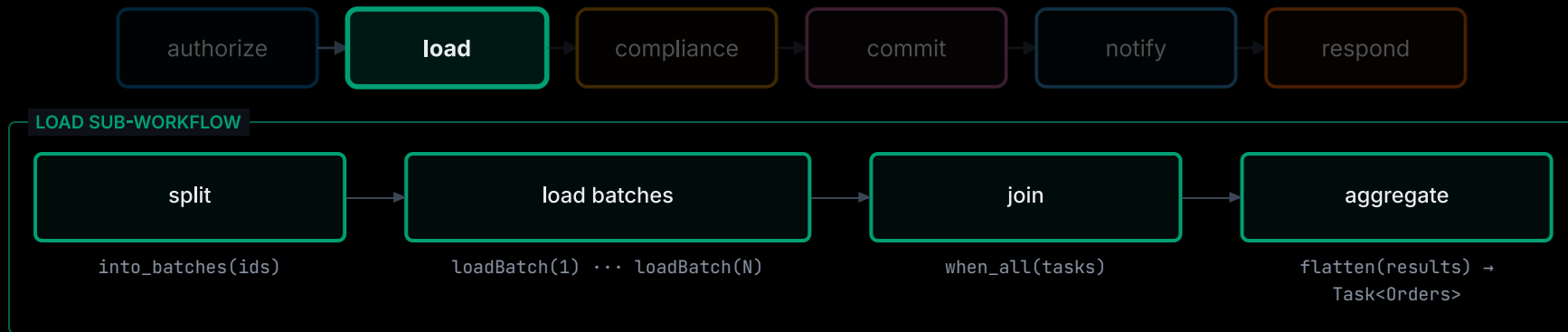
Async composition preserves structure

REQUEST WORKFLOW



Async composition preserves structure

REQUEST WORKFLOW



CONTROL

Each level reads as one flow.

Ordinary control flow stays inside; `co_await` connects levels.

DATA

Values cross boundaries; state stays local.

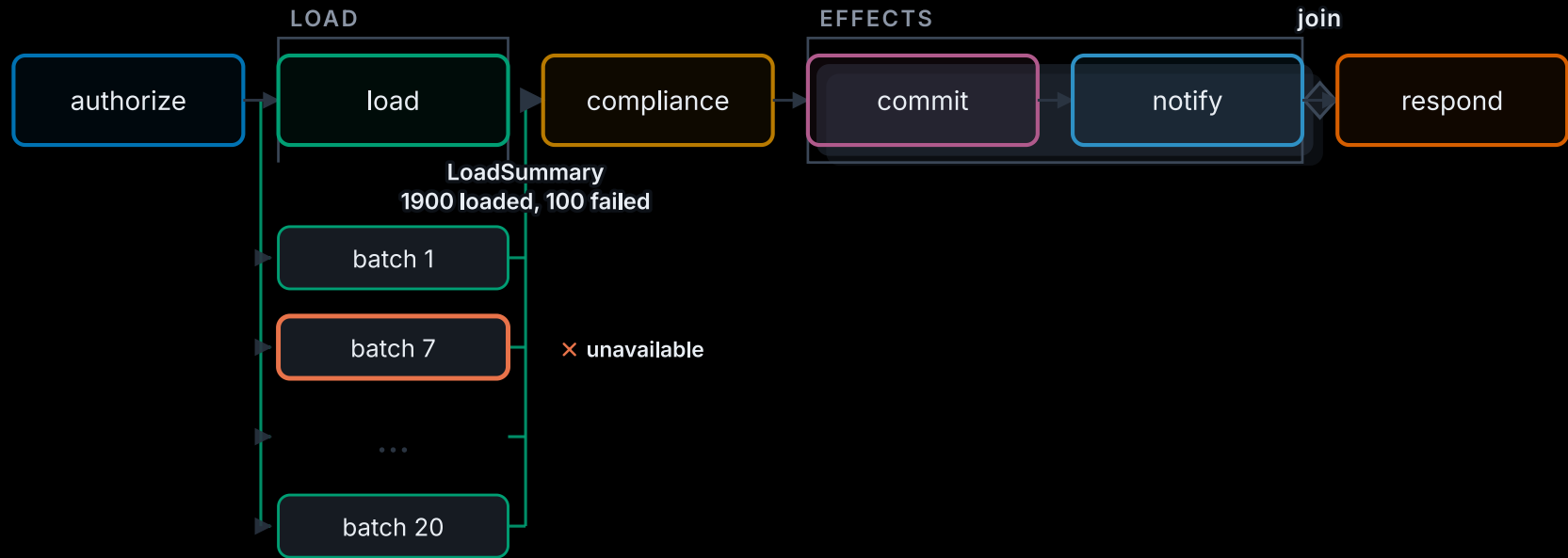
Inputs and results behave like ordinary function values.

COMPLETION

Many completions become one.

`when_all` joins the children; the parent awaits one operation.

Outcome policy: Allow partial success



each child returns expected<Batch, Error>

batch 7 completed. It completed with a failure value.

Fan-in answers "when", not "what it means"

when_all knows when every child is done. The parent still has to interpret the outcomes.

```

LoadBatch(1) → Task<std::expected<Batch, BatchError>>
LoadBatch(2) → Task<std::expected<Batch, BatchError>>
...
...
LoadBatch(20)→ Task<std::expected<Batch, BatchError>>

```

└─ when_all

```
result = 19 values + 1 error
```

when_all can answer "Are all children complete?"

It cannot decide whether these outcomes count as success.

Completion is mechanical. Success is semantic.

How should partial failure cross the boundary?

The return type can preserve or collapse partial outcomes.

```
Task<std::expected<???, LoadError>> loadOrders(OrderIds ids)
{
    auto results = co_await when_all(loadEach(ids));
    co_return interpret(results);
}
```

Same child outcomes. Different information crosses the boundary.

All-or-nothing Contract

```
??? → Orders

Task<std::expected<Orders, LoadError>>
loadOrders(OrderIds ids);
```

Partial failure becomes E

Summary Contract

```
??? → LoadSummary
struct LoadSummary {
    Orders loaded;
    LoadFailures failed;
};

Task<std::expected<LoadSummary, LoadError>>
loadOrders(OrderIds ids);
```

Partial failure remains visible in T

The workflow is finally boring again!

The contract carries the outcome. Ordinary control flow applies the policy.

```
Task<Response> activateOrders(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return makeResponse(authorized.error());

    auto loadedSummary = co_await loadOrders(request.orderIds);
    if (!loadedSummary) co_return makeResponse(loadedSummary.error());           // no summary

    if (!loadedSummary→failed.empty()) co_return makeResponse(loadedSummary→failed); // reject with details

    auto compliance = co_await checkCompliance(loadedSummary→loaded);
    if (!compliance) co_return makeResponse(compliance.error());

    auto effectsSummary = co_await writeAndNotifyEach(loadedSummary→loaded);
    if (!effectsSummary) co_return makeResponse(effectsSummary.error());         // no summary

    co_return makeResponse(effectsSummary→committed, effectsSummary→failed);    // report committed + failed
}
```

The workflow is finally boring again!

The contract carries the outcome. Ordinary control flow applies the policy.

```
Task<Response> activateOrders(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return makeResponse(authorized.error());

    auto loadedSummary = co_await loadOrders(request.orderIds);
    if (!loadedSummary) co_return makeResponse(loadedSummary.error());           // no summary

    if (!loadedSummary→failed.empty()) co_return makeResponse(loadedSummary→failed); // reject with details

    auto compliance = co_await checkCompliance(loadedSummary→loaded);
    if (!compliance) co_return makeResponse(compliance.error());

    auto effectsSummary = co_await writeAndNotifyEach(loadedSummary→loaded);
    if (!effectsSummary) co_return makeResponse(effectsSummary.error());         // no summary

    co_return makeResponse(effectsSummary→committed, effectsSummary→failed);    // report committed + failed
}
```

The workflow is finally boring again!

The contract carries the outcome. Ordinary control flow applies the policy.

```
Task<Response> activateOrders(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return makeResponse(authorized.error());

    auto loadedSummary = co_await loadOrders(request.orderIds);
    if (!loadedSummary) co_return makeResponse(loadedSummary.error());           // no summary

    if (!loadedSummary→failed.empty()) co_return makeResponse(loadedSummary→failed); // reject with details

    auto compliance = co_await checkCompliance(loadedSummary→loaded);
    if (!compliance) co_return makeResponse(compliance.error());

    auto effectsSummary = co_await writeAndNotifyEach(loadedSummary→loaded);
    if (!effectsSummary) co_return makeResponse(effectsSummary.error());         // no summary

    co_return makeResponse(effectsSummary→committed, effectsSummary→failed);    // report committed + failed
}
```

The workflow is finally boring again!

The contract carries the outcome. Ordinary control flow applies the policy.

```
Task<Response> activateOrders(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return makeResponse(authorized.error());

    auto loadedSummary = co_await loadOrders(request.orderIds);
    if (!loadedSummary) co_return makeResponse(loadedSummary.error());           // no summary

    if (!loadedSummary→failed.empty()) co_return makeResponse(loadedSummary→failed); // reject with details

    auto compliance = co_await checkCompliance(loadedSummary→loaded);
    if (!compliance) co_return makeResponse(compliance.error());

    auto effectsSummary = co_await writeAndNotifyEach(loadedSummary→loaded);
    if (!effectsSummary) co_return makeResponse(effectsSummary.error());         // no summary

    co_return makeResponse(effectsSummary→committed, effectsSummary→failed);    // report committed + failed
}
```

The workflow is finally boring again!

The contract carries the outcome. Ordinary control flow applies the policy.

```
Task<Response> activateOrders(Request request)
{
    auto authorized = co_await authorize(request);
    if (!authorized) co_return makeResponse(authorized.error());

    auto loadedSummary = co_await loadOrders(request.orderIds);
    if (!loadedSummary) co_return makeResponse(loadedSummary.error());           // no summary

    if (!loadedSummary→failed.empty()) co_return makeResponse(loadedSummary→failed); // reject with details

    auto compliance = co_await checkCompliance(loadedSummary→loaded);
    if (!compliance) co_return makeResponse(compliance.error());

    auto effectsSummary = co_await writeAndNotifyEach(loadedSummary→loaded);
    if (!effectsSummary) co_return makeResponse(effectsSummary.error());         // no summary

    co_return makeResponse(effectsSummary→committed, effectsSummary→failed);    // report committed + failed
}
```

Policy changes should land where they belong

Execution policy

MECHANISM ↓

HIGHER LAYER

what work should happen



hide local detail

LOWER LAYER

how the work runs

Outcome policy

FACTS ↑

HIGHER LAYER

what the result means



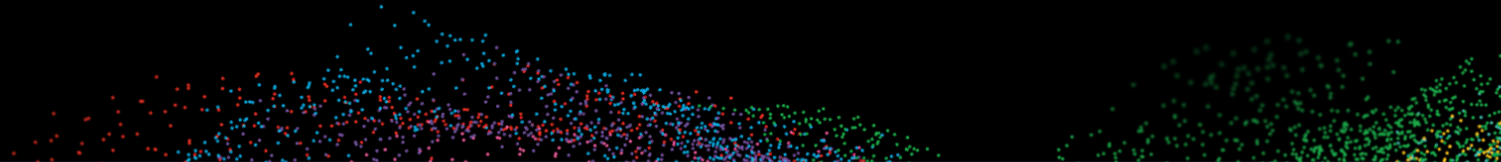
preserve decision context

LOWER LAYER

what actually happened

Keep the blast radius small.

Put each decision at the narrowest layer that has enough context to own it.



Workflow structure is locally visible again

	Sync	Callback-based	Graph-based	Coroutines + <code>expected</code>
Control flow <i>How do I know what runs next?</i>	source order / stack	continuation chain	declared edges	top-to-bottom code
Data flow <i>Where do values live between steps?</i>	locals / returns	callback args / captured state	shared <code>Context</code>	locals / <code>co_await</code> results
Completion <i>How do I know when I can move on?</i>	function return	callback + counter/latch	runtime-managed join	<code>co_await</code> / <code>when_all</code>
Error flow <i>How does failure propagate?</i>	return code / throw	callback arg / convention	node result / edge policy	typed alternative outcome (<code>E</code>)

Task<std::expected<T, E>>

Task

When is the operation complete?

std::expected

What does the result mean?

Async complexity is inevitable. Scattered complexity is not.

QUESTIONS

Bloomberg

Engineering

Thank You!

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Engineering

Bloomberg